

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Codebase esistenti

Un repository mai visto

Claude comincia leggendo il codice: che cosa trova, e che cosa no

- Nel Modulo 4, Claude Code lavora nel repository: legge, agisce, controlla. Qui, il codice che **c'è già**: un repository che Claude non ha mai visto — o, per chi gestisce sistemi, uno **script ereditato** — e che spesso nessuno ha spiegato nemmeno a voi
- Per Anthropic, Claude comincia ogni compito **leggendo il codice**, e i modelli attuali lo fanno **molto bene**
- Per orientarsi, la documentazione propone di partire da una **panoramica**, «give me an overview of this codebase», e poi di passare a domande **via via più strette**, su parti precise
- Ciò che **non trova** è ciò che nel repository non c'è mai stato: per esempio, una procedura di deploy che vive **nella testa di una persona**
- Prima di scrivere qualcosa per Claude, due domande: potrebbe **trovarlo** leggendo il codice? Andrebbe davvero **a cercarlo**? Se la risposta è sì a entrambe, **lasciatelo fuori**

Chiedere bene

Il file, il sintomo, e com’è il «risolto»

- La pagina delle best practice: se **nominate il file**, **descrivete il sintomo** e dite fin dall’inizio **com’è il «risolto»**, di solito vi risparmiate una tornata di correzioni in seguito

vaga: l’esempio della documentazione, tradotto

```
migliora questa codebase
```

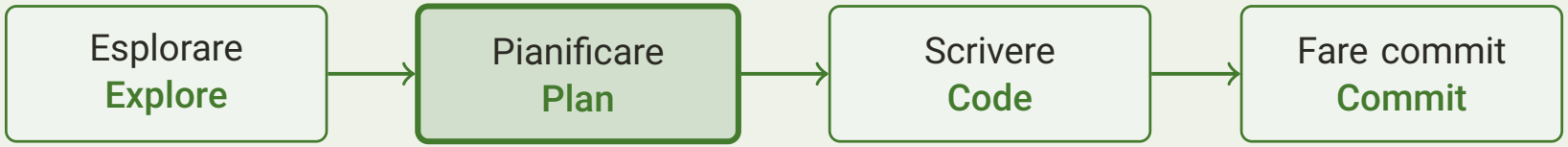
specifica: dalla pagina delle best practice, tradotto

```
gli utenti segnalano che il login fallisce dopo il timeout
della sessione. guarda il refresh del token in src/auth/,
scrivi un test che fallisce e lo riproduce, poi correggilo
```

- La richiesta vaga fa **esplorare** il codice in largo; quella specifica fa lavorare Claude con **poche letture** di file
- Nella seconda ci sono tutte e tre le cose: il **posto**, `src/auth/`; il **sintomo**, il login che fallisce dopo il timeout; il **«risolto»**, un test che riproduce il guasto, e poi la correzione

Esplorare, pianificare, scrivere, fare commit

Il flusso che l’Academy di Anthropic chiede di portarsi a casa, e plan mode



- Il corso «Claude Code 101» dell’Academy di Anthropic: se ve ne portate a casa **una cosa sola**, sia questo flusso. Senza, i più chiedono subito a Claude di scrivere codice, e poi servono **più correzioni di rotta**
- Per i compiti **complessi** — modifiche ampie, più strade valide, scelte di architettura, più file — **plan mode**: Claude legge, cerca, esegue comandi di esplorazione e propone un approccio da approvare, invece di rilavorare dopo; non modifica nulla **finché non approvate**
- La revisione del piano è il posto migliore per **correggere la rotta**: nessuna riga di codice è ancora scritta. E per essere sicuro dei risultati, Claude deve sapere con chiarezza **com’è il «corretto»**

Refactoring sicuro

A piccoli passi, con lo stesso comportamento, e una strada per tornare indietro

- La ricetta della documentazione di Claude Code: rifattorizzare **mantenendo lo stesso comportamento**, e farlo a **piccoli incrementi verificabili**
- Partire da uno **stato git pulito**, e fare spesso **commit di checkpoint**: lo raccomandano i team di Anthropic, per annullare con facilità le modifiche sbagliate
- Dopo ogni passo, i **test**: qualcosa che Claude può eseguire e che risponde «passa» o «fallisce». Con quello può **andare avanti da solo**; e i test dicono se il comportamento è rimasto quello
- Il plugin `code-modernization`, per modernizzare il codice: «Nothing edits your legacy source», il sorgente legacy **non si modifica**; e si comincia da **un modulo pilota**, non da tutto
- A voce: che cosa vi serve per **fidarvi** di un refactoring fatto da un agente?

Test su codice senza test

Prima i test che mancano, poi un giudice provato

- La ricetta della documentazione: prima trovare le funzioni che **nessun test copre**, poi far aggiungere i test dei **casi limite**
- Scrivendo i test, dice la documentazione, Claude guarda i **test che ci sono**, per adeguarsi a stile, framework e asserzioni già in uso
- Un test che decide se una modifica è giusta è un **giudice**. Nelle migrazioni del codice, scrive Anthropic, senza un giudice solido mancano una **condizione d'uscita** e una misura del successo
- E il giudice va **provato**: deve passare sul codice **originale** e fallire su codice **rotto apposta**. Se passa anche lì, non prova niente
- Poi chiedete le **prove**: il comando che Claude ha eseguito, e il suo output

Prima esplorare, poi pianificare, poi modificare: con un test che dice se il comportamento è rimasto quello.

Documentare un modulo opaco

Ciò che il codice fa, messo per iscritto, e un controllo che non è un’opinione

- La ricetta della documentazione: trovare le funzioni **senza documentazione**, farla scrivere, e alla fine **controllare** che rispetti gli standard del progetto
- Nelle migrazioni, scrive Anthropic, il **vecchio codice** fa da ottima **specifica** per il modello
- La revisione sia **avversaria**, la verifica **meccanica**: a fare da arbitro siano degli script — un compilatore, un diff, una suite di test
- Nel laboratorio, la traccia (b) fa proprio questo, su un modulo del gestionale: la documentazione, i test che ne fissano il comportamento, una proposta di refactoring

Dal vivo: un file mai visto

Nel deck di pratica, il riscaldamento: che cosa guardare, tre domande e una quarta

- Subito dopo, nel deck di pratica, l'esercitazione **M4-E01**: `src/Copertura/VerificaCopertura.php`, un file del gestionale PHP di CiaspoLink che nessuno vi ha spiegato. Claude Code lo spiega, ne scrive un **test** e lo esegue, poi vi mostra il **comando** con il suo output. Per Anthropic, scrivendo i test Claude guarda **quelli che ci sono**: lo si mette alla prova
- **Legge prima di scrivere?** Prima del test, apre `VerificaCopertura.php` e i test che ci sono? Per Anthropic, Claude comincia ogni compito **leggendo il codice**
- **Esegue, o dice «dovrebbe funzionare»?** Claude si ferma quando il lavoro **sembra** finito; qui ha un controllo da eseguire, il comando dei test, che risponde «passa» o «fallisce»
- **Chiede prima di toccare una regola?** Se il test non passa, corregge il test o vi propone di cambiare `VerificaCopertura.php`? Una regola del dominio la decidete **voi**
- E la quarta: **l'agente non firma**. La spiegazione è utile solo se qualcuno la verifica: per Anthropic i test automatici aiutano a verificare la funzionalità, ma la **revisione di una persona** resta cruciale